

1. PROJECT OVERVIEW

The Smart Health Access System is a mobile application designed to close the gap between patients and healthcare providers in communities where clinic visits are difficult to arrange, medical histories are scattered across paper files, and appointment scheduling depends on long queues or phone calls that often go unanswered. The system gives patients a single place to register, search for doctors by specialty, book and track appointments, and view their own medical history, while giving doctors a structured way to manage their daily schedule and update patient records, and giving administrators the oversight tools needed to manage users, manage doctors, and generate reports on system usage.

The system directly supports United Nations Sustainable Development Goal 3 (Good Health and Well-Being), which calls for ensuring healthy lives and promoting well-being for all ages, including the target of achieving universal health coverage and access to quality essential healthcare services. By digitising appointment booking, medical record-keeping, and health reminders, the Smart Health Access System reduces missed appointments, shortens the time between a patient recognising a symptom and seeing a doctor, and gives both patients and clinicians a shared, accurate record of care over time.

1.1 Problem Statement

In many under-resourced healthcare settings, patients have no reliable way to know which doctors are available, when they can be seen, or what was discussed at their last visit. Appointments are frequently booked by walking into a clinic or by phone, which disadvantages patients who cannot easily travel or who call outside office hours. Medical records are often kept on paper, which makes it difficult for a doctor to review a patient's history quickly, increases the risk of lost information, and prevents patients from having any independent copy of their own health data.

1.2 Proposed Solution

The Smart Health Access System addresses this problem with a single mobile application serving three distinct user roles. Patients can register, search for doctors by specialty, book appointments with a clear view of availability, receive automated reminders for appointments and medication, and view or download their own medical history. Doctors can view and manage their daily appointment schedule, update patient records after a consultation, and review a patient's full history before treating them. Administrators oversee the platform itself: managing user and doctor accounts and generating reports that give a picture of overall system usage and clinic load.

1.3 Target Users

User Role	Description	Primary Needs
Patient	Member of the public seeking healthcare services	Book appointments, view records, get reminders
Doctor	Licensed medical practitioner using the platform	Manage schedule, update records, review patient history
Administrator	Platform/clinic staff managing the system	Manage users and doctors, generate usage reports

Table 1: Target user roles for the Smart Health Access System

2. Software Development Life Cycle (SDLC)

The Smart Health Access System was developed using the Iterative and Incremental SDLC model. This model was chosen over a strict Waterfall approach because the project's UI/UX design, prototype, and documentation all needed to evolve together: early feedback on the use case diagram shaped the screens that were designed in Figma, and early prototype screens in turn revealed gaps in the original requirements that had to be revisited. An iterative model allows each phase to be revisited in light of what was learned in later phases, rather than treating requirements as fixed from day one, which better reflects how the project actually progressed across the seven-week build.

Rather than completing requirements, design, implementation, and testing once in a single long pass, the project was broken into short increments, each of which produced a working, demonstrable piece of the system: first the core navigation and authentication screens, then the patient-facing booking and records features, then the doctor and admin modules, with each increment reviewed and refined before the next began. This made it possible to catch a misunderstanding about, for example, what a doctor's dashboard needed to show, well before the entire prototype was finished.

2.1 Why Iterative and Incremental Over Other Models

Three alternative models were considered before the Iterative and Incremental model was selected:

Model	Strengths	Why Not Chosen
Waterfall	Clear phases, strong documentation	Requirements cannot be fixed upfront; no working product until final stage — too risky for a seven-week timeline
Spiral	Formal risk management, suits safety-critical systems	Designed for large-budget, high-risk projects requiring dedicated risk analysts; overhead exceeds the scope of this academic project

RAD	Very fast prototyping	Sacrifices architecture rigour and documentation depth required by the brief
Iterative & Incremental	Working software at every increment, continuous feedback, flexible to change	Selected — best fit for evolving requirements, seven-week constraint, and Digital Public Good reusability goals

Table 2: SDLC model comparison and justification

2.2 Phases of the Iterative and Incremental Model Applied to This System

Phase 1: Planning and Requirements Gathering

This phase defined the problem the system would solve, identified the three user roles (Patient, Doctor, Administrator), and translated the project brief's requirements into a concrete feature list: registration and login, doctor search, appointment booking, medical records, notifications, and the three admin functions of managing users, managing doctors, and generating reports. The SDG alignment with Goal 3 was confirmed at this stage, anchoring every later design decision to a real-world health access problem rather than a generic CRUD application.

Phase 2: System and Architecture Design

With requirements confirmed, the three-tier system architecture (presentation, application, and data layers) was designed, along with the use case diagram showing how each actor interacts with the system, the class diagram modelling the core entities (User, Patient, Doctor, Admin, Appointment, MedicalRecord, Notification), and a sequence diagram detailing the step-by-step flow of the most critical use case — booking an appointment. These diagrams were treated as living documents: the class diagram was revised once the prototype screens made clear that appointments needed a status field.

Phase 3: Incremental Implementation (Prototype Build)

Implementation proceeded in four increments rather than as one pass:

- Increment 1: Splash screen, login screen, and registration (Create Account) flow — establishing the authentication foundation
- Increment 2: Patient home dashboard, Find a Doctor screen, and Book Appointment screen — the core patient journey
- Increment 3: Medical Records screen and Notifications screen — completing the patient-facing feature set
- Increment 4: Doctor Dashboard and Admin Dashboard — the professional and management role modules

Each increment was reviewed against the use case diagram before the next began, which is the defining characteristic of an incremental build: working, demonstrable software at the end of every short cycle, not just at the very end of the project.

Phase 4: Testing and Review

Each increment was tested for internal consistency. Once the Book Appointment screen was built, it was verified against the Doctor Dashboard to confirm that a booking made by a patient would appear correctly in the doctor's daily appointment list. The Medical Records screen was checked against the sequence diagram to confirm data fields aligned correctly. Because testing happened after every increment rather than only at the end, design mistakes were caught while they were still cheap to fix.

Test Case	Action	Expected Result	Status
TC-01	Patient registers with valid credentials	Account created, redirected to dashboard	Pass
TC-02	Patient books appointment with available doctor	Appointment confirmed, appears on Doctor Dashboard	Pass
TC-03	Patient attempts to double-book same slot	Scheduling conflict detected, booking rejected	Pass
TC-04	Doctor updates patient record post-consultation	Record updated and visible in Medical Records screen	Pass
TC-05	Admin generates system usage report	Report displays correct aggregated statistics	Pass

Table 3: Sample test cases with outcomes

Phase 5: Deployment and Documentation

The final phase consolidated all increments into the complete prototype, finalised the GitHub repository structure, applied the MIT License, and produced this report. Although the system described here is a prototype rather than a live deployment, this phase mirrors what a real deployment phase would involve: packaging the finished increments, documenting how the system works, and preparing it for others to use, extend, or contribute to.

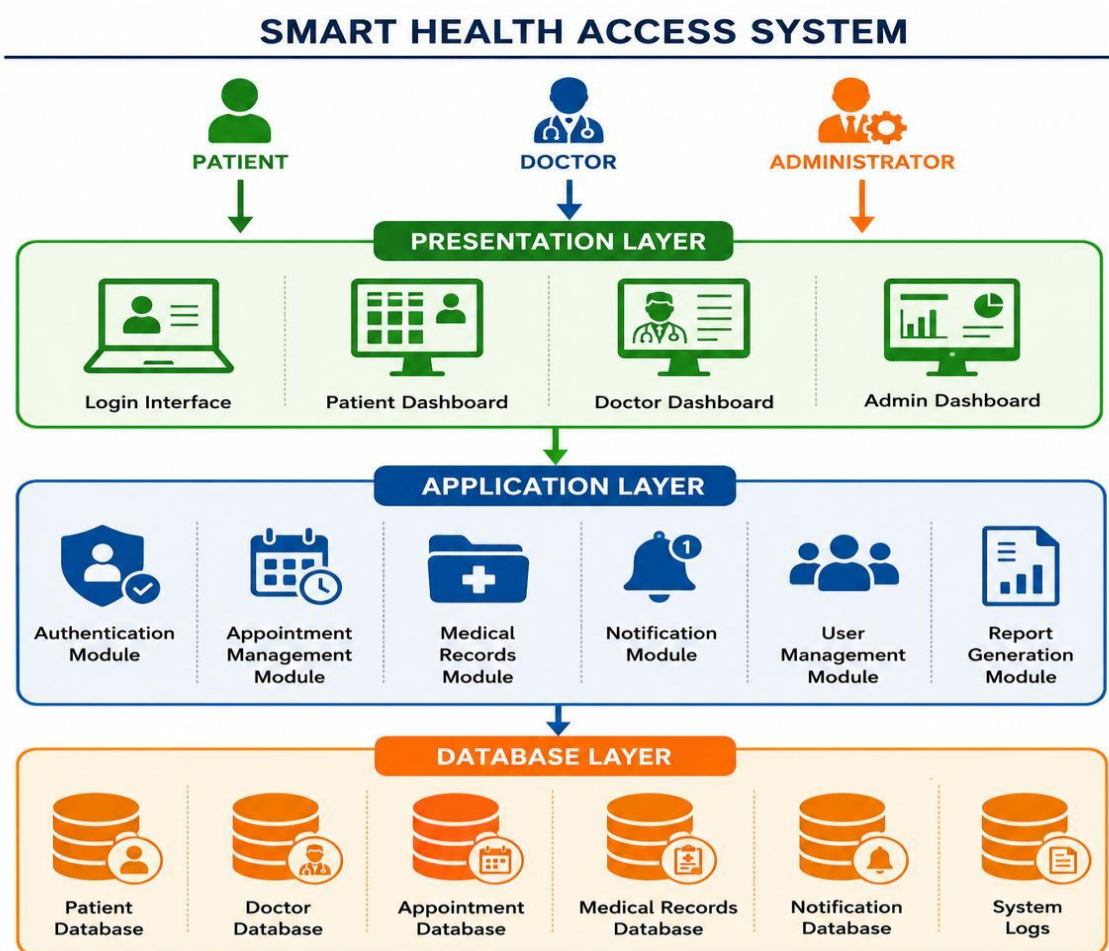
2.3 Why Iterative and Incremental Fits This Project

A Digital Public Good is, by definition, meant to be reviewed, reused, and improved by others, which is much easier when the system was built in clearly separated, working increments rather than as one monolithic effort. The iterative model also matched the realistic constraint of a seven-week timeline shared across research, architecture, design, prototyping, and documentation: it allowed design and documentation work to begin on early increments while later features were still being built, instead of forcing every requirement to be finalised before any screen could be designed.

3. SYSTEM ARCHITECTURE DESIGN

The Smart Health Access System follows a three-tier architecture, separating the mobile client, the application logic, and the data storage into distinct layers. This separation keeps the mobile application lightweight and focused purely on presentation, while all business rules — such as preventing two patients from booking the same doctor at the same time — live in a single, testable application layer rather than being duplicated across client code.

3.1 Architecture Diagram



*Figure 1: Three-tier system architecture of the Smart Health Access System
[Architecture diagram — see Figma/GitHub for visual rendering]*

The architecture diagram below (Figure 1) illustrates the three-tier structure with the Presentation Layer at the top serving the three user roles, the Application Layer containing six functional modules in the middle, and the Database Layer at the base storing six distinct data collections.

3.2 Layer Descriptions

Presentation Layer (Mobile Client)

Built with React Native, this layer renders the screens shown in Section 5 and is organised into three role-based modules:

- Patient App Module: registration, login, booking, records, notifications, profile
- Doctor App Module: appointment list, schedule updates, patient history
- Admin App Module: user management, doctor management, reports

A shared component library provides common UI elements — buttons, form inputs, and the bottom navigation bar — so the three modules stay visually consistent.

Application Layer (API & Business Logic)

The application layer exposes a RESTful API gateway that handles all client requests. Six functional modules implement the system's business rules:

Module	Responsibility
Authentication Module	Handles registration, login, session management, and role-based access control
Appointment Management	Manages booking creation, conflict detection, status updates, and schedule queries
Medical Records Module	Stores and retrieves patient history, prescriptions, lab results, and downloadable reports
Notification Module	Triggers appointment reminders, medication alerts, and health updates
User Management Module	Admin tools for creating, editing, and deactivating patient and doctor accounts
Report Generation Module	Aggregates usage statistics and produces admin-facing summary reports

Table 4: Application layer modules and responsibilities

Database Layer

MongoDB stores all persistent data across six collections: Patient Database, Doctor Database, Appointment Database, Medical Records Database, Notification Database, and System Logs.

3.3 Component Interaction

When a patient books an appointment, the mobile client sends a request to the API gateway, which passes it through the validation and middleware layer for authentication and input checks before the business logic service checks for a scheduling conflict and writes the new appointment to MongoDB. Once the database confirms the write, the API returns a success response that the client uses to display a booking confirmation. This same pattern — client to API gateway to business logic to database and back — applies to every feature in the system, keeping the architecture predictable even as new features are added. Section 4.3 walks through this exact flow as a sequence diagram.

4. UML DIAGRAMS

Three complementary UML diagrams were developed to model the Smart Health Access System from different angles: a use case diagram capturing what each actor can do, a class diagram capturing the system's underlying data structures and how they relate, and a sequence diagram capturing the step-by-step interaction behind the system's most important function — booking an appointment.

4.1 Use Case Diagram

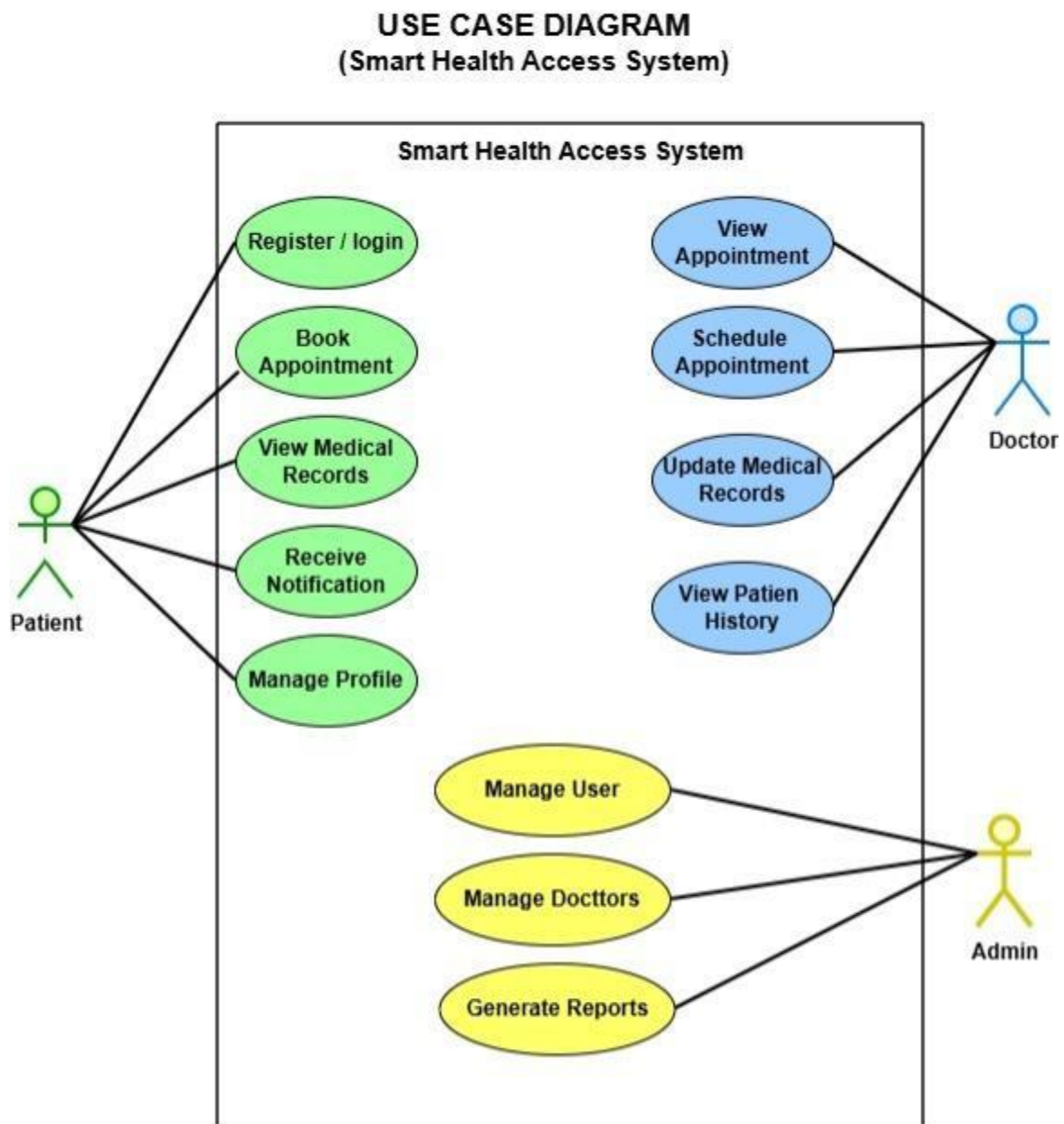


Figure 2: Use Case Diagram of the Smart Health Access System [See attached diagram / GitHub /diagrams folder]

The use case diagram identifies three actors — Patient, Doctor, and Administrator — and the functions each can perform. A patient can register or log in, book an appointment, view medical records, receive notifications, and manage their profile. A doctor can view and schedule appointments, update medical records, and view a patient's history. An administrator can manage users, manage doctors, and generate reports. Separating these use cases by actor early in the project made it straightforward to later organise the prototype itself into three matching app modules.

4.2 Class Diagram

The class diagram models the seven core entities of the system and the relationships between them. All entities inherit from a base User class, which holds common attributes shared across roles.

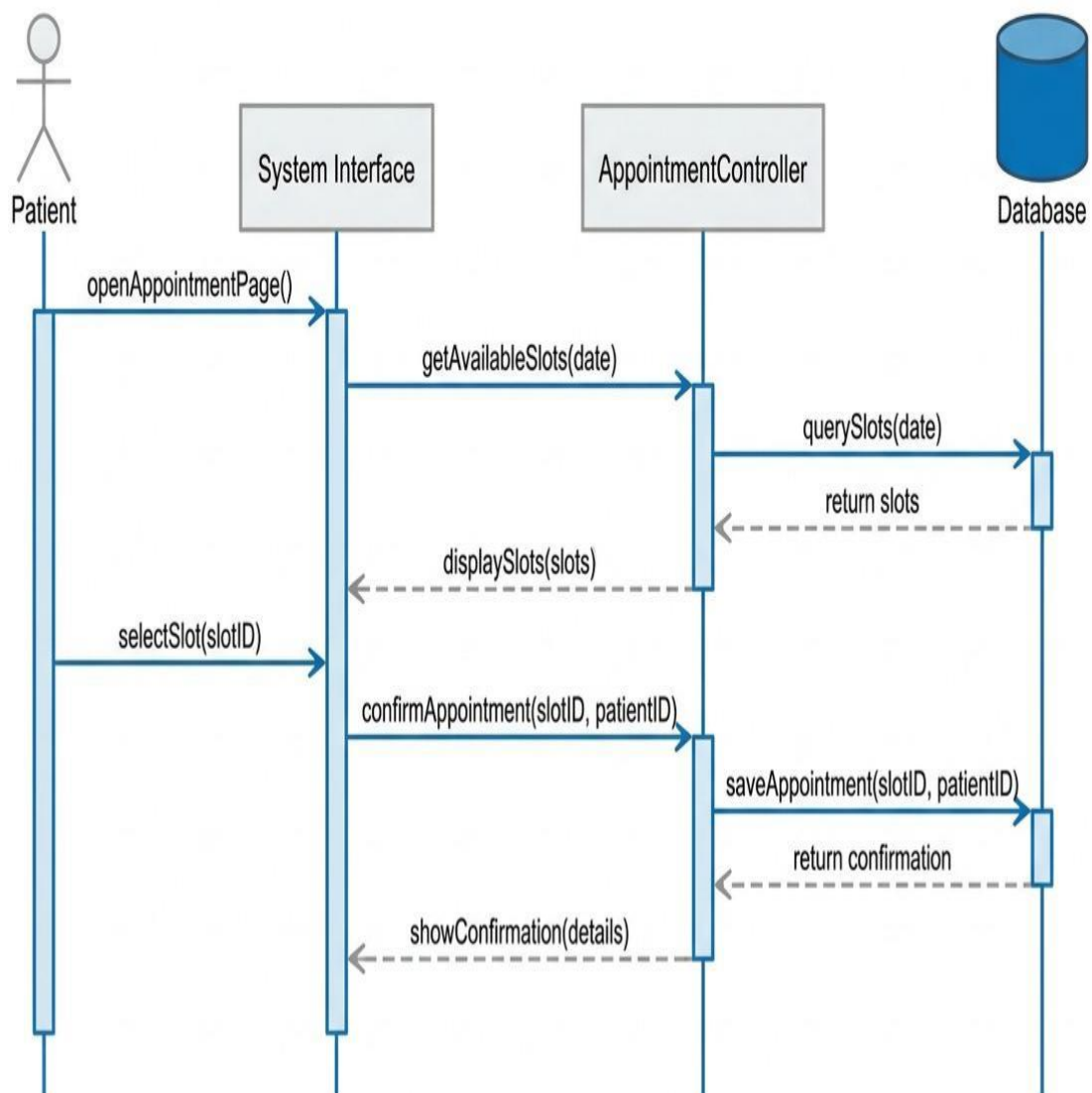
Class	Key Attributes	Key Relationships
User	userID, name, email, phone, passwordHash, role	Base class; extended by Patient, Doctor, Admin
Patient	patientID, dateOfBirth, bloodGroup, address	Has many Appointments; has many MedicalRecords
Doctor	doctorID, specialty, licenseNumber, availability, rating	Has many Appointments; updates MedicalRecords
Admin	adminID, accessLevel	Manages User accounts; generates Reports
Appointment	appointmentID, date, time, reason, status (Scheduled/Completed/Cancelled)	Belongs to Patient; belongs to Doctor
MedicalRecord	recordID, diagnosis, prescription, labResults, notes, createdAt	Belongs to Patient; created/updated by Doctor
Notification	notificationID, type, message, timestamp, isRead	Belongs to User; triggered by Appointment events

Table 5: Class diagram entities, attributes, and relationships

Note: The Appointment entity includes a status field (Scheduled, Completed, Cancelled) that was added during the iterative prototype review phase, when testing revealed that the Doctor Dashboard needed to distinguish between upcoming and past appointments. This is a concrete example of the iterative model enabling requirement refinement mid-build.

4.3 Sequence Diagram — Book Appointment

UML Sequence Diagram: Book Appointment Flow - Smart Health Access System



The sequence diagram traces the Book Appointment use case across all three architectural layers, showing precisely how a single patient action becomes a database write and a confirmation message:

- Patient opens the Book Appointment screen
- Mobile app calls `getAvailableSlots(date)` on the System Interface
- AppointmentController queries MongoDB: `querySlots(date)`
- Database returns available slots; interface displays them to the patient
- Patient selects a doctor, date, and time, then confirms
- App calls `confirmAppointment(slotID, patientID)`
- AppointmentController checks for scheduling conflicts before calling `saveAppointment()`
- MongoDB acknowledges the write and returns confirmation
- API returns success; app displays booking confirmation to patient
-

5. PROTOTYPE DEVELOPMENT (UI/UX DESIGN)

The prototype was designed in Figma as a high-fidelity, role-aware mobile experience covering all three user types identified in the use case diagram. Ten key screens were designed — exceeding the brief's minimum requirement — in order to give a complete walkthrough of the patient journey from first launch through booking and records, alongside the doctor and admin views that support it. A consistent visual language — a green-and-white palette evoking health and trust, rounded cards, and a fixed bottom navigation bar for patients — was applied across every screen so the application feels like one coherent product rather than a set of disconnected mockups.

5.1 Design Principles Applied

- Consistency: the same header style, button shape, and colour palette recur on every screen so users always know where they are
- Clarity: each dashboard (Patient, Doctor, Admin) leads with the numbers and actions that role needs most, rather than a generic menu

- Accessibility: high-contrast text on white cards, large tappable buttons, and icons paired with text labels rather than icons alone
- Role separation: patients, doctors, and admins each get a dashboard tailored to their use cases instead of one shared screen with hidden permissions

5.2 GUI Figma Screens

Screen 1: Welcome / Splash Screen

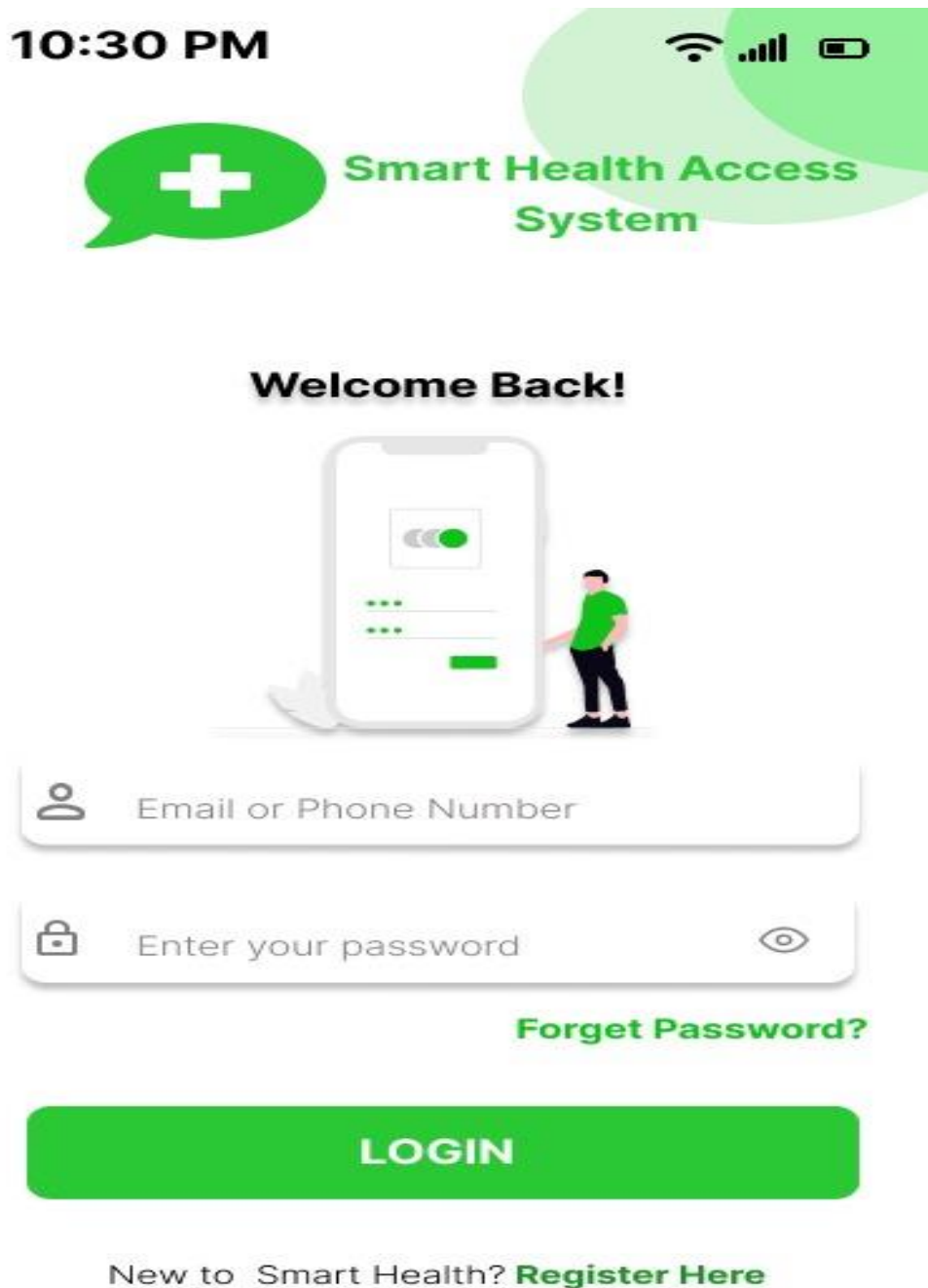
The first screen a new user sees introduces the system's purpose — 'Healthcare Anytime, Anywhere' — with supporting illustration and a single Get Started call-to-action, deliberately limiting the first impression to one decision rather than a wall of options.



Figure 4: Welcome / Splash Screen

Screen 2: Login Screen

Returning users sign in with an email or phone number and password, with a visible password-reveal toggle and a Forgot Password link. A clear 'Register Here' link at the bottom routes new users to account creation without forcing them to hunt for it.



The login screen features a status bar at the top with the time 10:30 PM, signal strength, and battery level. Below the status bar is a green speech bubble icon with a white plus sign, followed by the text "Smart Health Access System". The main heading "Welcome Back!" is centered. Below the heading is an illustration of a smartphone with a login form on its screen, and a small figure of a person standing next to it. The login form consists of two input fields: "Email or Phone Number" and "Enter your password". The password field has a visible password-reveal toggle (an eye icon). Below the password field is a link "Forgot Password?". At the bottom is a large green button labeled "LOGIN". Below the button is the text "New to Smart Health? Register Here".

10:30 PM

Smart Health Access System

Welcome Back!

Email or Phone Number

Enter your password

Forgot Password?

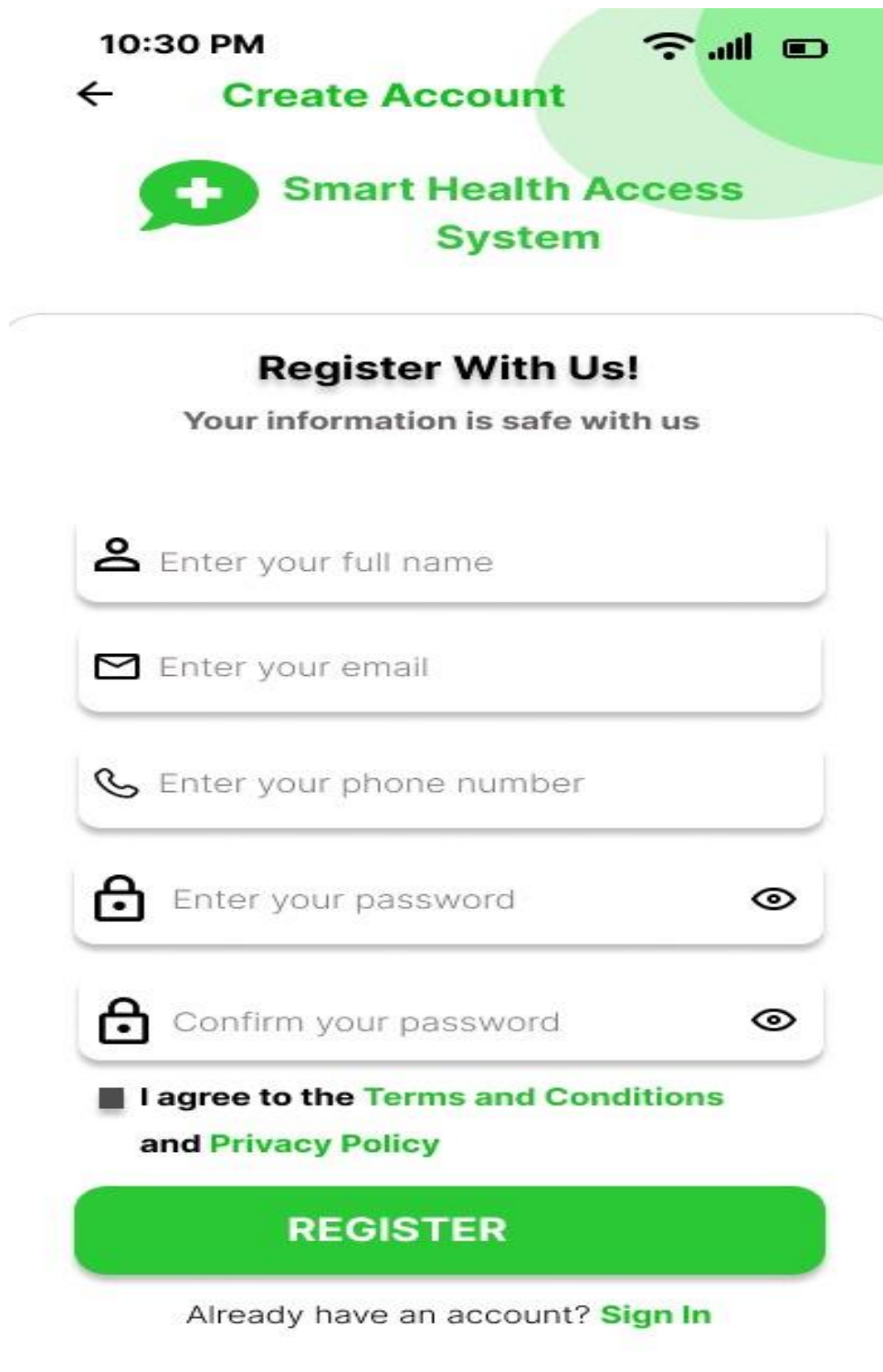
LOGIN

New to Smart Health? Register Here

Figure 5: Login Screen

Screen 3: Create Account Screen

New users provide their full name, email, phone number, and password, with password confirmation and an explicit Terms and Conditions / Privacy Policy checkbox to support informed consent — which matters in a health-data context.



The screenshot shows a mobile application interface for creating an account. At the top, the status bar displays '10:30 PM' and icons for Wi-Fi, cellular signal, and battery. Below the status bar is a navigation bar with a back arrow and the title 'Create Account'. The main header features a green speech bubble icon with a white plus sign and the text 'Smart Health Access System'. The main content area is titled 'Register With Us!' with the subtitle 'Your information is safe with us'. Below this are five input fields: 'Enter your full name' (with a person icon), 'Enter your email' (with an envelope icon), 'Enter your phone number' (with a phone icon), 'Enter your password' (with a lock icon and a toggle eye icon), and 'Confirm your password' (with a lock icon and a toggle eye icon). Below the input fields is a checkbox labeled 'I agree to the Terms and Conditions and Privacy Policy'. At the bottom is a large green button labeled 'REGISTER'. Below the button is the text 'Already have an account? Sign In'.

10:30 PM

← Create Account

Smart Health Access System

Register With Us!
Your information is safe with us

Enter your full name

Enter your email

Enter your phone number

Enter your password

Confirm your password

☐ I agree to the Terms and Conditions and Privacy Policy

REGISTER

Already have an account? [Sign In](#)

Figure 6: Create Account Screen

5.3 Patient Experience

Screen 4: Patient Home Dashboard

The patient's home screen greets them by name and surfaces the two numbers that matter most at a glance — upcoming appointments and new notifications — above a Quick Actions row giving one-tap access to booking, records, notifications, and profile.

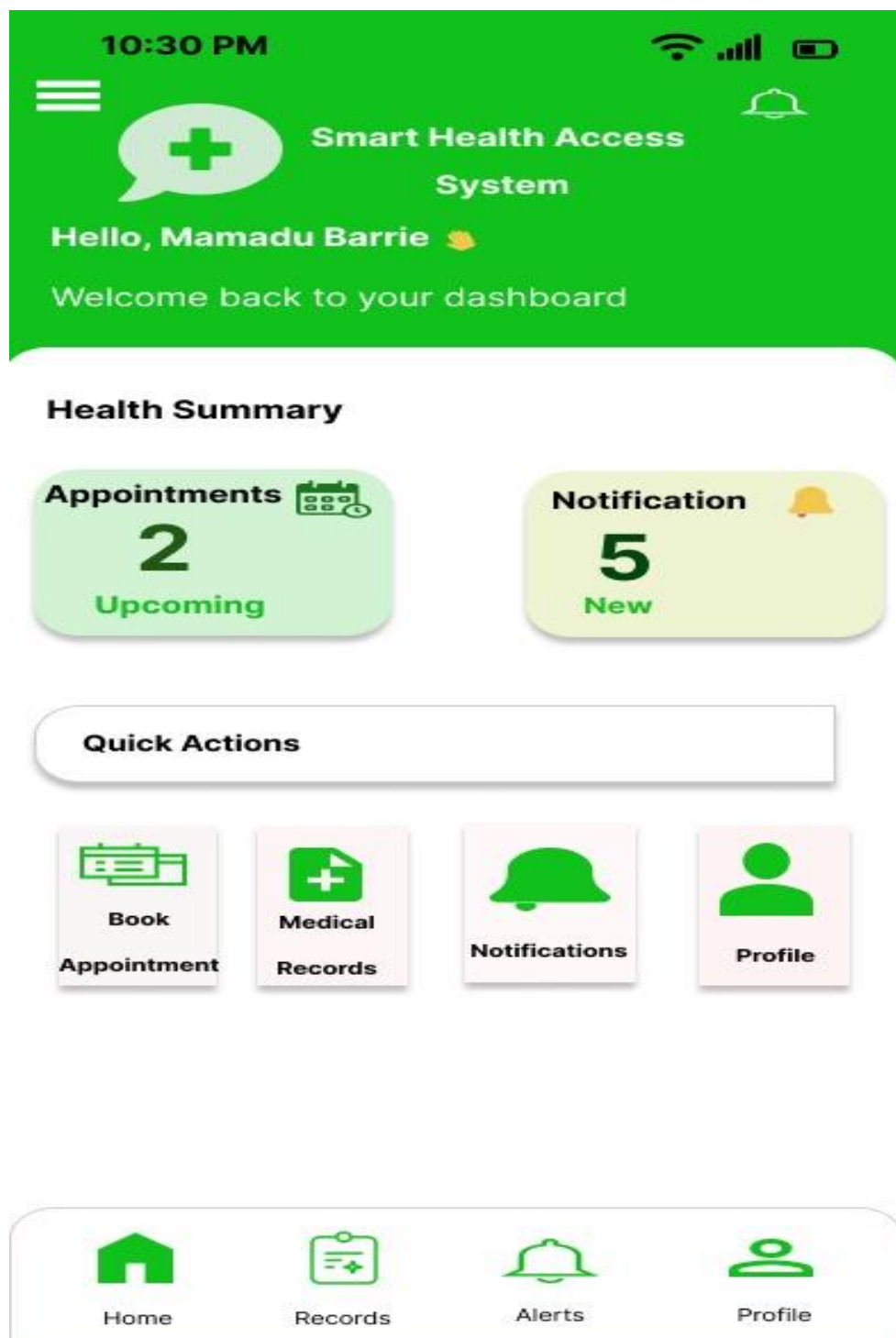


Figure 7: Patient Home Dashboard

Screen 5: Find a Doctor Screen

Patients search by doctor name or specialty and see each doctor's specialty, star rating, review count, and live availability status before tapping View Profile, so the choice of doctor is informed rather than blind.

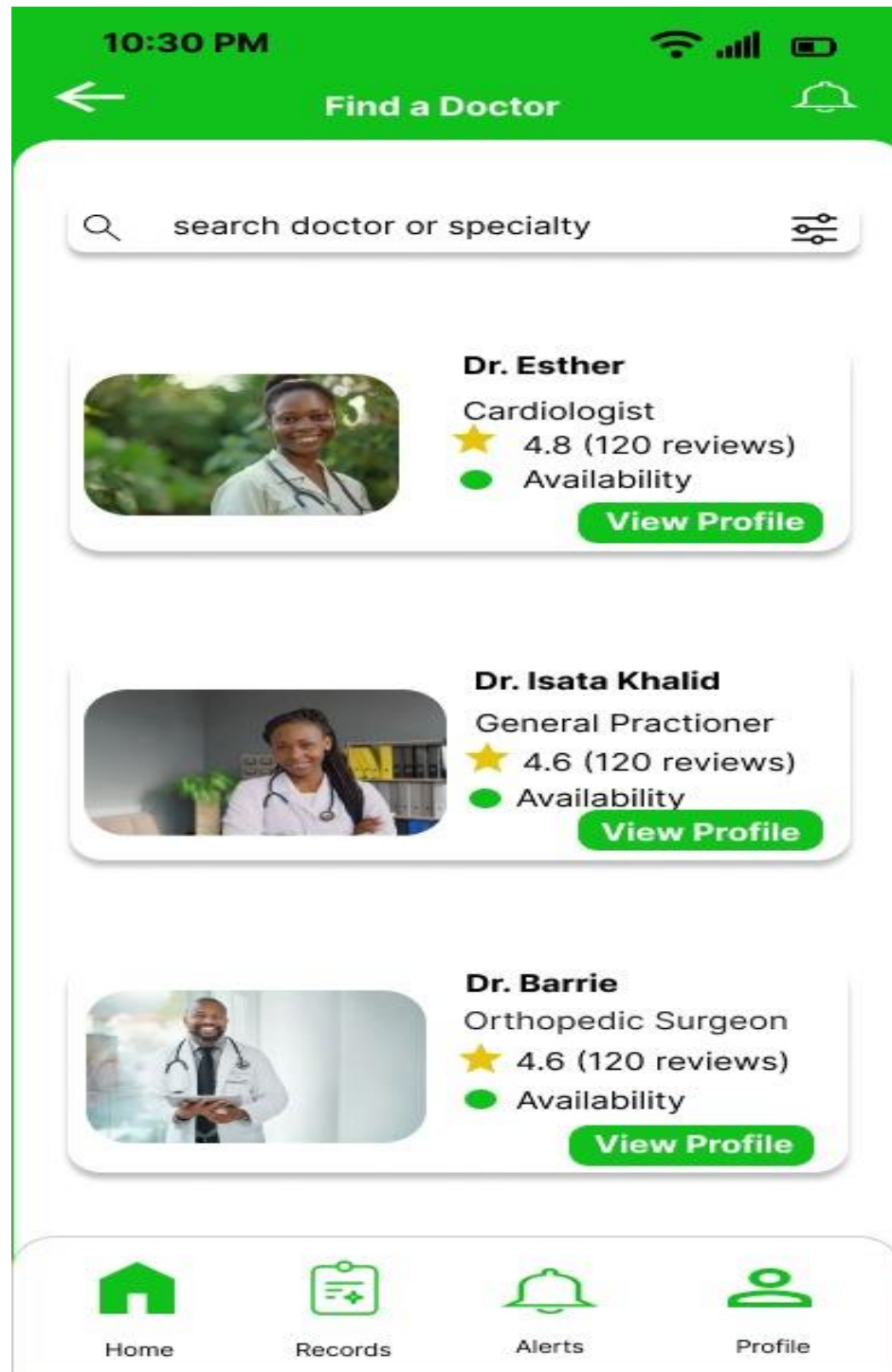


Figure 8: Find a Doctor Screen

Screen 6: Book Appointment Screen

Once a doctor is chosen, the patient selects a date and time and enters a free-text reason for the visit before confirming, giving the doctor useful context before the consultation even begins.

10:30 PM

Book Appointment

Select Doctor

Dr. Barrie (Orthopedic Surgeon) ▼

Select Date

4 June 2026

Select Time

10:30 AM ▼

Reason for Visit

Deep aching in my hips, knees, and shoulder that makes movement difficult.....

CONFIRM BOOKING

Figure 9: Book Appointment Screen

Screen 7: Medical Records Screen

Patients view their own history organised into History, Prescriptions, Lab Results, and Reports tabs, with a Download Records (PDF) action that lets a patient carry a portable copy of their data to a different clinic if needed — a key digital-public-good principle of patient data ownership.



Figure 10: Medical Records Screen

Screen 8: Notifications Screen

Reminders are grouped under All, Appointments, Reminders, and Alerts filters, covering appointment reminders, medication reminders, and broader health updates such as a nearby health camp, keeping patients engaged with their care between visits rather than only at appointment time.

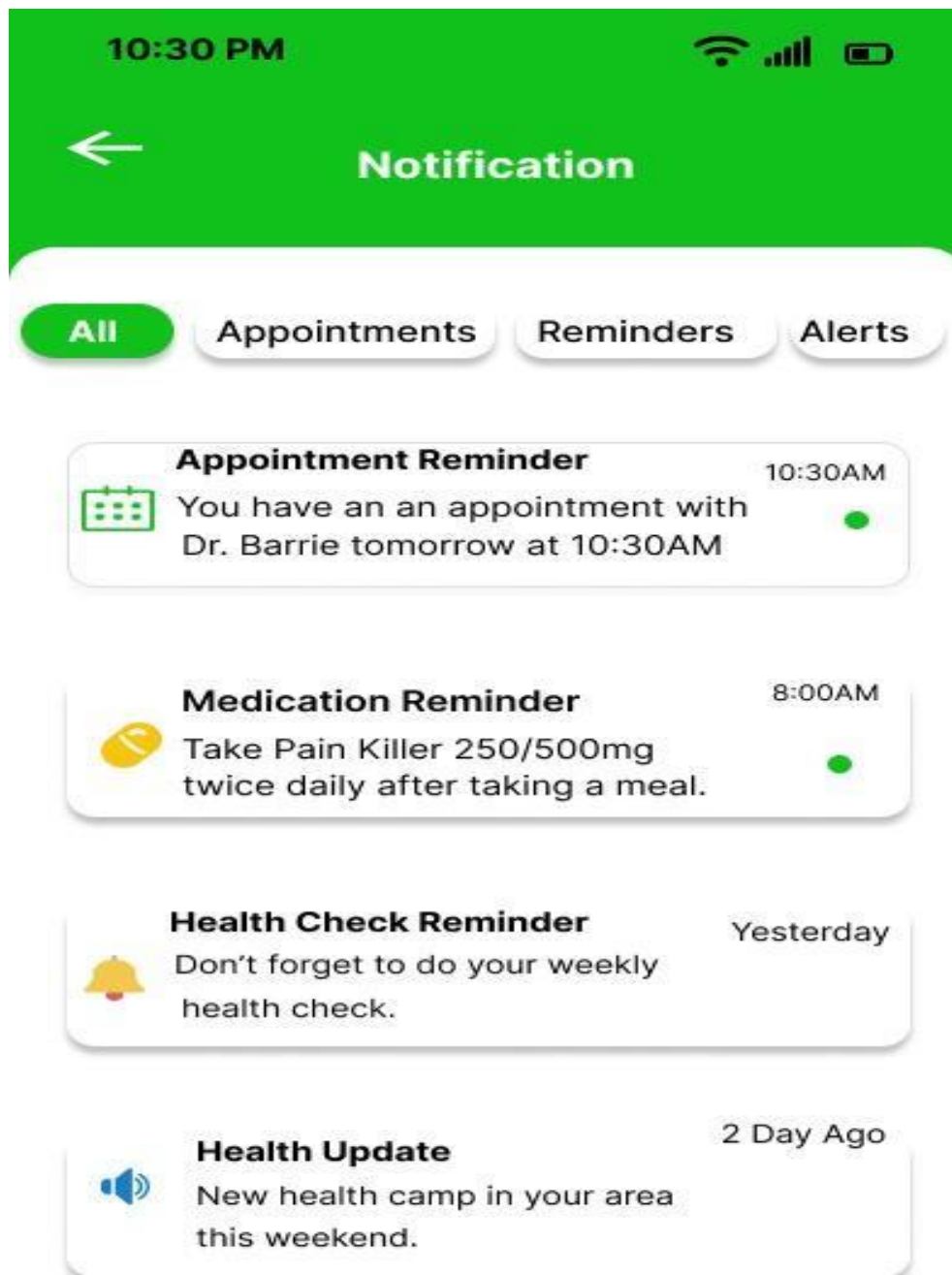


Figure 11: Notifications Screen

5.4 Doctor Experience

Screen 9: Doctor Dashboard

On logging in, a doctor immediately sees today's appointment count, how many patients are waiting, and how many consultations are already completed, followed by a same-day appointment list and direct actions to view the full schedule or update patient records.

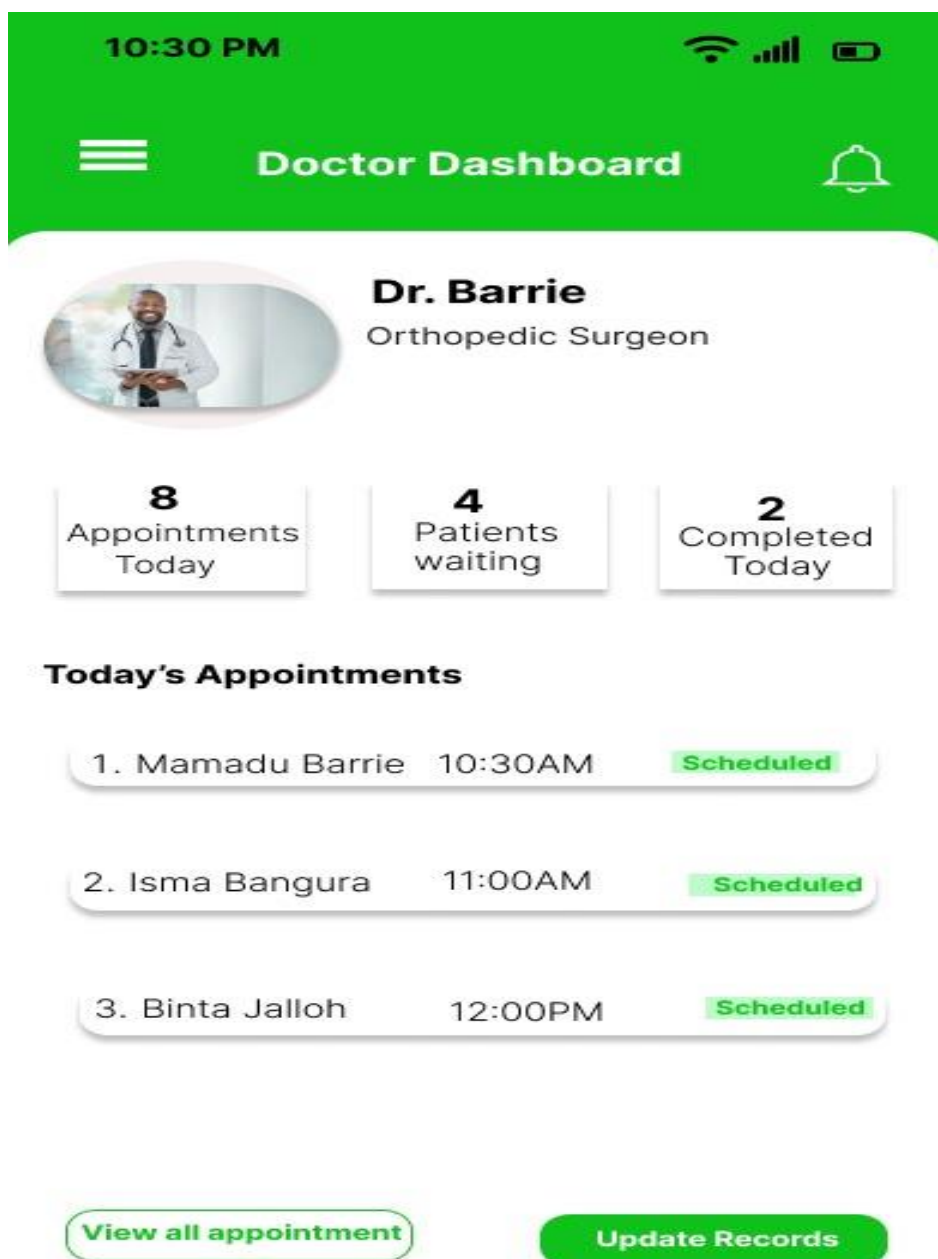


Figure 12: Doctor Dashboard

5.5 Administrator Experience

Screen 10: Admin Dashboard

The administrator's dashboard summarises total users, total doctors, total appointments, and today's view count, with direct access to the three management functions defined in the use case diagram: Manage Users, Manage Doctors, and Generate Reports, plus System Settings.

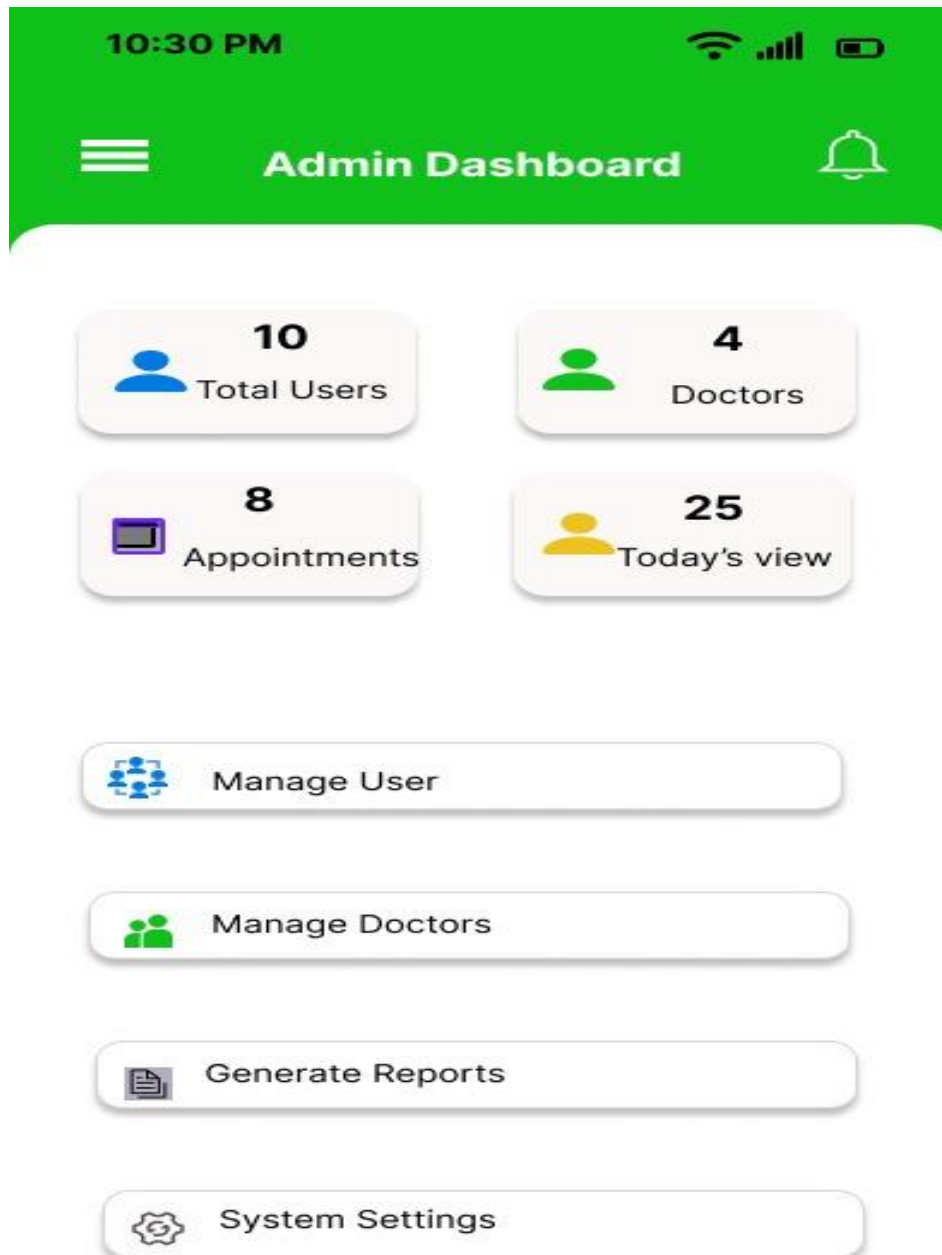


Figure 13: Admin Dashboard

5.6 Mapping Screens to Use Cases

Every screen maps directly back to a use case from Figure 2, demonstrating that each documented requirement has a working interface behind it — not just a diagram.

Use Case (Figure 2)	Corresponding Screen(s)
Register / Login	Create Account (Screen 3), Login (Screen 2)
Book Appointment	Find a Doctor (Screen 5), Book Appointment (Screen 6)
View Medical Records	Medical Records (Screen 7)
Receive Notification	Notifications (Screen 8)
Manage Profile	Patient Home Dashboard — Profile quick action (Screen 4)
View / Schedule Appointment	Doctor Dashboard (Screen 9)
Update Medical Records	Doctor Dashboard — Update Records action (Screen 9)
View Patient History	Doctor Dashboard (Screen 9)
Manage Users / Manage Doctors / Generate Reports	Admin Dashboard (Screen 10)

Table 6: Traceability between use cases and prototype screens

6. GIT & GITHUB INTEGRATION

Version control was managed through a dedicated GitHub repository created for the Smart Health Access System, used to store and track all project artefacts and to support collaboration between team members across the project's seven-week build.

6.1 Repository Structure

Folder / File	Contents
Software Documentation/	Project report (this document), SDLC explanation, licence file
Software/diagrams/	Use case, class, sequence, architecture, Gantt, and network diagrams
UI-UX/prototype/	Exported Figma screens and prototype link
README.md	Project overview, setup instructions, and SDG alignment statement
LICENSE	MIT Licence text

Table 7: GitHub repository structure

6.2 Collaboration Workflow

Work was committed incrementally in line with the Iterative and Incremental SDLC model described in Section 2: each completed screen, diagram, or document section was committed with a descriptive message rather than committing the entire project at once, which keeps the commit history itself a useful record of how the project evolved. Branches were used to separate design work from documentation work so that one did not block the other, and changes were merged back into the main branch once reviewed.

- main branch: holds the stable, reviewed version of all project artefacts

- feature/prototype-screens branch: used while building and refining the Figma exports
- feature/documentation branch: used while drafting and revising this report
- Commits reference the relevant project task (e.g. 'Add Book Appointment screen — Task E') for traceability against the Gantt chart in Section 7

6.3 README and Onboarding

The repository's README.md gives a short project description, the SDG 3 alignment statement, setup instructions for running the client and server locally, and a link to the live Figma prototype, so that the repository is usable by someone encountering the project for the first time — not only by its original authors. This is part of what makes a GitHub repository genuinely support a Digital Public Good rather than merely store files.

7. PROJECT PLAN

The project was planned across the seven weeks allotted for the Final Project (Weeks 5 to 11), with each week assigned to one or more of nine tasks. A Gantt chart shows the planned duration and timing of each task, and a network diagram shows the dependencies between tasks and identifies the critical path — the sequence of tasks that determines the minimum possible length of the entire project.

7.1 Gantt Chart

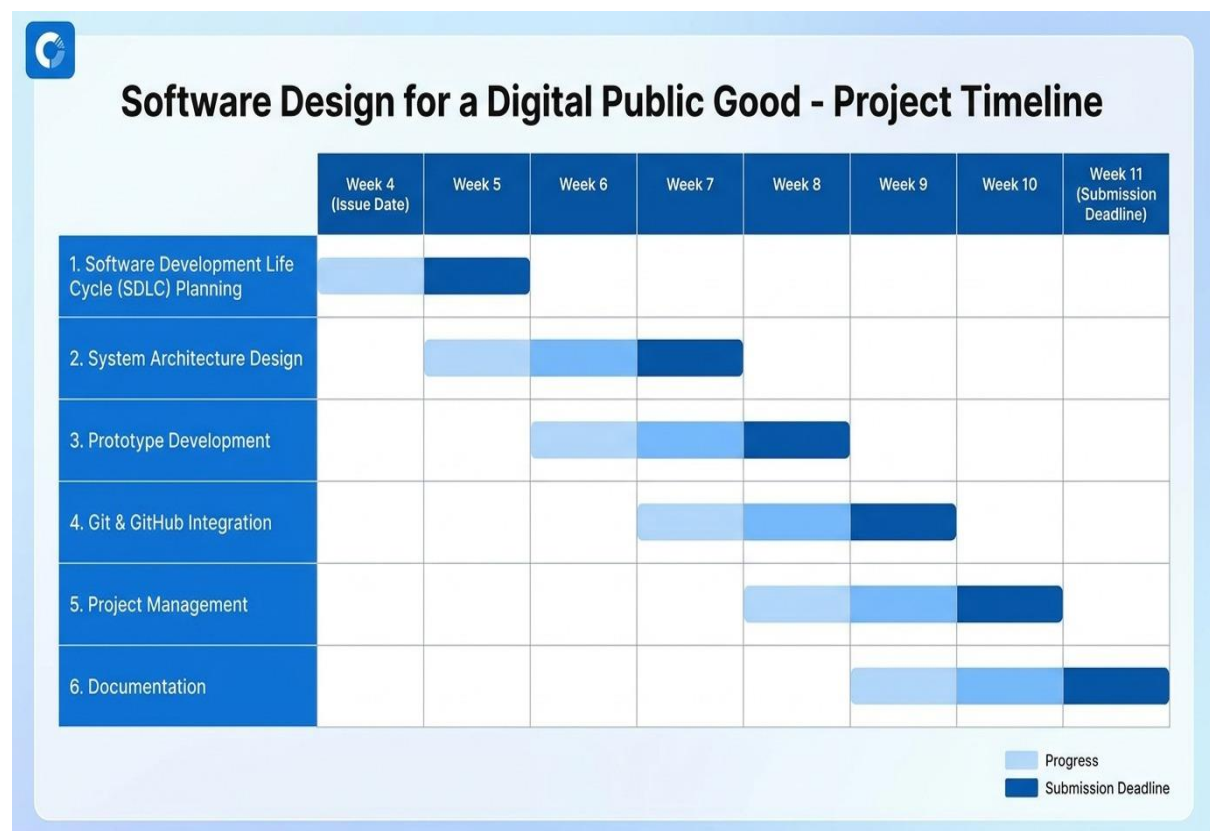


Figure 14: Project Gantt Chart, Weeks 4–11 [See attached Gantt chart / GitHub /diagrams folder]

Two tasks — GitHub Setup & System Architecture (Task C) and UML Diagrams (Task D) — were scheduled in parallel during Week 6, since architecture decisions and the class/sequence diagrams informed each other and did not need to wait on one another to begin. Every other task proceeds sequentially: each depends on the deliverable from the task before it. For example, the UI/UX Design in Week 7 depends on the system architecture from Week 6 being finalised first.

7.2 Network Diagram and Critical Path

Software Engineering Project Network Diagram (PERT/CPM)

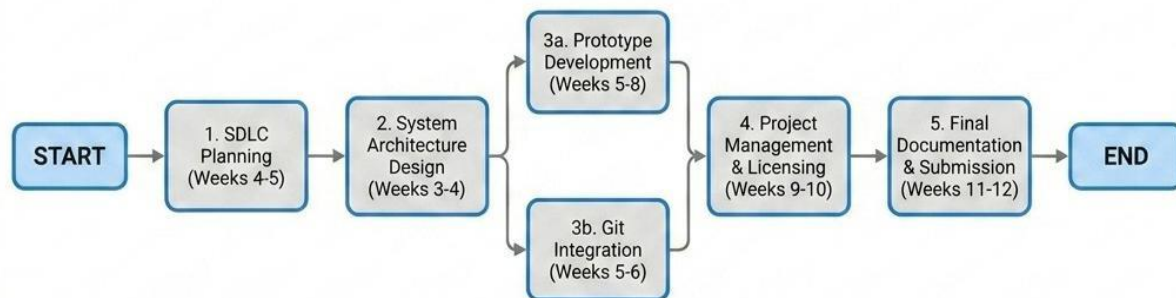


Figure 15: Network Diagram (PERT/CPM) showing task dependencies and critical path [See attached network diagram / GitHub /diagrams folder]

The critical path runs through: Research (A) → SDLC Planning (B) → GitHub Setup & Architecture (C) → UI/UX Design (E) → Prototype Build (F) → Licence & SDG Review (G) → Documentation (H) → Final Review (I). This is the longest chain of dependent tasks in the project, so any delay along this path delays the whole project. The UML Diagrams task (D) has slack: it could run a few days later without affecting the submission deadline, since its only downstream dependency is the UI/UX Design task it feeds into alongside Task C.

7.3 Task Summary

Task	Description	Duration	Critical Path
A	SDG & Requirements Research	2 days	Yes
B	SDLC Model & Iteration Planning	5 days	Yes
C	GitHub Setup & System Architecture	5 days	Yes
D	UML Diagrams	2 days	No (slack)
E	UI/UX Design — 10 Figma Screens	2 weeks	Yes
F	Functional Prototype Build	4 days	Yes
G	Licence Selection & SDG Review	2 days	Yes
H	Report Documentation	3 weeks	Yes
I	Final Review & Submission	1 day	Yes
	Total	8 weeks	

Table 8: Project task summary with critical path identification

8. LICENCE, DESIGN QUALITY & SDG RELEVANCE

8.1 Licence Selection

The Smart Health Access System is released under the MIT Licence. The MIT Licence was chosen because it is short, widely understood, and places almost no restriction on how others may use, modify, or distribute the software, requiring only that the original copyright notice and licence text be preserved in any copy or substantial portion of the software.

For a project whose explicit goal is to function as a Digital Public Good — freely reusable by other developers, clinics, or NGOs facing the same access-to-healthcare problem — a permissive licence like MIT removes legal friction that could otherwise prevent adoption, which a more restrictive or copyleft licence could introduce. The full licence text is stored in the repository's LICENSE file at the root of the GitHub repository, and a short licensing notice is included in the README so that anyone discovering the project understands their usage rights immediately.

8.2 Attribution of External Resources

Several third-party resources were used in building the prototype and are credited here in line with their own usage terms:

- Figma (2026) was used to design and prototype all ten screens shown in Section 5
- Illustration assets used on the Welcome and Get Started screens are stock vector illustrations licensed for design use
- Icons used throughout the interface follow common open icon-set conventions (calendar, bell, profile, and similar symbols) rather than any single proprietary icon library
- No copyrighted text, branding, or imagery belonging to an existing commercial health platform was reproduced; all screen content, copy, and sample data (such as the doctor names and sample medical record entries) were created specifically for this project

8.3 Design Quality (UI/UX)

Design quality was treated as a first-class requirement rather than an afterthought, consistent with the marking criteria in the project brief. A single colour palette — primarily green to evoke health and trust, paired with white space and rounded cards — is applied consistently across all ten screens. Navigation follows a small number of clear patterns: a fixed bottom tab bar for the patient's four core sections (Home, Records, Alerts, Profile), a consistent header with a back arrow and notification bell on every secondary screen, and large, clearly labelled primary action buttons (Login, Register, Confirm Booking) that are never ambiguous about what they will do.

Information hierarchy was also considered deliberately: each dashboard leads with the two or three numbers a user most needs — a patient's upcoming appointments and new notifications; a doctor's appointments today, patients waiting, and completions; an admin's total users, doctors, appointments, and today's views — before offering secondary actions, rather than presenting every possible option with equal visual weight.

8.4 SDG RELEVANCE AND REAL-WORLD IMPACT

The system's alignment with SDG 3 is not a label applied after the fact — it shapes specific design decisions throughout the product:

Feature	SDG 3 Target Addressed	Real-World Impact
Appointment booking & reminders	Universal health coverage (3.8)	Reduces missed appointments and access barriers
Medication reminders (Notifications screen)	Treatment adherence	Supports multi-day medication schedules without clinical follow-up

Downloadable PDF medical records	Patient data ownership	Enables portability across clinics; supports continuity of care
Doctor patient-history view	Quality of care (3.8)	Ensures continuity even with high staff turnover
MIT Licence & open-source repository	Equitable access to digital health tools	Enables free reuse by NGOs and clinics in low-resource settings

Table 9: SDG 3 alignment mapped to system features and real-world impacts

Taken together, these features describe a realistic, deployable improvement to how a small clinic or a network of community health workers could operate today, rather than a speculative or purely academic exercise — which is the real-world problem-solving value the project brief asks the system to demonstrate.

9. CONCLUSION

This report has presented the complete design and prototype of the Smart Health Access System, a Digital Public Good aligned with Sustainable Development Goal 3. The project was developed using an Iterative and Incremental SDLC model, supported by a three-tier system architecture, modelled through use case, class, and sequence UML diagrams, planned through a Gantt chart and network diagram, and brought to life through a ten-screen, high-fidelity Figma prototype covering the Patient, Doctor, and Administrator roles. The project is version-controlled on GitHub under an Iterative and Incremental development workflow and released under the MIT Licence to support its goal of being freely reusable by others working on the same real-world problem.

Beyond satisfying the project's assessment criteria, the Smart Health Access System demonstrates how a digital public good can be approached end-to-end: starting from a genuine access-to-healthcare problem, working through a defensible architecture and data model, and arriving at a prototype that a patient, doctor, or administrator could plausibly use today.

10. REFERENCES

- Figma (2026) Figma: The collaborative interface design tool. Available at: <https://www.figma.com> (Accessed: 10 June 2026).
- GitHub (2026) GitHub: Build and ship software on a single, collaborative platform. Available at: <https://github.com> (Accessed: 10 June 2026).
- International Organization for Standardization (ISO) (2011) ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE). Geneva: ISO.
- Object Management Group (OMG) (2017) Unified Modeling Language (UML), Version 2.5.1. Needham, MA: OMG.
- Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. 9th ed. New York: McGraw-Hill.
- Sommerville, I. (2016) Software Engineering. 10th ed. Boston: Pearson.
- United Nations (2026) Goal 3: Ensure healthy lives and promote well-being for all at all ages. Available at: <https://sdgs.un.org/goals/goal3> (Accessed: 10 June 2026).